AI-Powered RAG-Based Department Information Chatbot

Project Report submitted in partial fulfillment of the requirements for the award of the degree of **Master of Science (Five Year Integrated)** in **Computer Science (Artificial Intelligence & Data Science)**

by

ASWIN SURESH

(Register No : 80522004)

Under the Supervision of,

Dr. JEENA KLEENANKANDY

Assistant Professor

Department of Computer Science

Department of Computer Science

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

APRIL 2025

Certificate

This is to certify that the project report for ***21-805-0805 : PROJECT*** under the topic ***AI-Powered RAG-Based Department Information Chatbot*** is a record of work carried out by ***Aswin Suresh (80522004)***, in partial fulfillment of the requirements for the award of degree in ***Master of Science (Five Year Integrated)*** in ***Computer Science (Artificial Intelligence & Data Science)*** of Cochin University of Science and Technology (CUSAT), Kochi. The project report has been approved as it satisfies the academic requirements in respect of the eighth semester prescribed for the Master of Science (Five Year Integrated) in Computer Science degree.

Faculty Member in-charge

Dr. Jeena Kleenankandy
Assistant Professor
Department of Computer Science
CUSAT

Dr. Madhu S. Nair
Professor and Head
Department of Computer Science
CUSAT

Declaration

I hereby declare that the project report entitled **"AI-Powered RAG-Based Department Information Chatbot"** submitted as part of the semester eight curriculum for the Integrated MSc in Computer Science with specialization in Artificial Intelligence and Data Science at Cochin University of Science and Technology is my original work. This written submission represents my ideas in my own words. Where others' ideas and words have been included, I have adequately cited and referenced the original source. I declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in my submission. I understand that any violation of the above will cause disciplinary action by the Institute and can also evoke penal action from the source which has not been properly cited or from whom proper permission has not been taken when needed.

. . . . . .

ASWIN SURESH
Roll No.: 80522004
Date: 20 April 2026
Place: CUSAT, Kalamassery

Acknowledgement

I express my profound gratitude to the Almighty for granting me the strength and determination to pursue this eighth semester mini project. I am deeply thankful to my guide Dr. Jeena Kleenankandy, Assistant Professor, Department of Computer Science, CUSAT, for her invaluable guidance, expertise, and continuous support throughout this project. Her timely feedback and encouragement have been instrumental in shaping both the technical direction and the quality of this work.

I also extend my heartfelt appreciation to Dr. Madhu S. Nair, Professor and Head of the Department of Computer Science, CUSAT, for his administrative support and for fostering a research-friendly environment in the department. My gratitude goes to all faculty members and staff of the department for their cooperation and encouragement throughout my academic journey.

I am also grateful to my family and friends for their unwavering moral support and motivation. Their encouragement has been a constant source of inspiration that helped me overcome every challenge in completing this project.

Abstract

Academic departments maintain large volumes of information spanning faculty profiles, academic programmes, research activities, placements, fee structures, laboratory facilities, and administrative details. This information is frequently scattered across multiple web pages and documents, making it difficult for students, faculty, and visitors to retrieve specific details efficiently. This project develops an AI-powered Retrieval-Augmented Generation (RAG) based chatbot tailored for the Department of Computer Science, Cochin University of Science and Technology (CUSAT). The system ingests structured department information from PDF documents and indexes it into a persistent vector database using ChromaDB with BGE embeddings. When a user poses a query, the system performs hybrid retrieval combining semantic vector search with keyword-based matching to fetch the most relevant context. A Large Language Model (LLM) hosted on the Groq inference platform (LLaMA-4-Scout-17B) synthesizes a coherent, natural-language response from the retrieved context. The system features a fully functional web-based user interface built with HTML, CSS, and JavaScript, served through a FastAPI backend. Key features include persistent multi-turn chat history stored in SQLite, a staff panel for dynamic document management, mobile-responsive design with a collapsible sidebar, and Markdown rendering in chat output. The chatbot successfully retrieves faculty information, programme details, placement statistics, research areas, fee structures, PhD admission procedures, and more, demonstrating high retrieval accuracy and natural response quality. This system serves as a 24/7 accessible, scalable, and easily updatable information assistant for the department.

Contents

Certificate i
Declaration ii
Acknowledgement iii
Abstract iv

1 Introduction **1**
1.1 Overview 1
1.2 Motivation 2
1.3 Objectives 2
1.4 Problem Statement 3
1.5 Scope and Limitations 4

2 Literature Review **5**
2.1 Introduction 5
2.2 Key Studies and Related Work 6
2.3 Summary 8

3 Methodology **9**
3.1 Introduction 9
3.2 Knowledge Base Construction 11
3.3 Document Ingestion and Chunking 11
3.4 Embedding Model and Vector Storage 14
3.5 Hybrid Retrieval Mechanism 15
3.6 LLM Integration and Response Generation 17
3.7 Backend API Design 18
3.8 Frontend Interface Design 19

4 Results and Analysis **20**
4.1 Introduction 20
4.2 System Performance 21
4.3 Query Testing Results 21
4.4 Discussion 23
4.5 Limitations of Current Implementation 24

5 Conclusion **25**
5.1 Summary 25
5.2 Impact and Applications 26

6 Future Work **27**
6.1 Completed Features 27
6.2 Future Directions 28

System Screenshots **29**
.1 Technology Stack Summary 29

List of Figures

3.1 End-to-end system architecture of the RAG-based chatbot. 10
3.2 Document ingestion and chunking pipeline. 13
3.3 Hybrid retrieval mechanism. 16

List of Tables

3.1 Embedding model and vector storage configuration. 14
3.2 LLM configuration for response generation. 17

4.1 System performance characteristics. 21
4.2 Query testing results for representative questions. 22

1 Complete technology stack. 29

Chapter 1

Introduction

1.1 Overview

The rapid advancement of Artificial Intelligence and Natural Language Processing has opened transformative possibilities in information retrieval and human-computer interaction. One of the most impactful applications of these technologies is the development of intelligent chatbots that can understand natural language queries and respond with contextually relevant, accurate information. In an academic environment, the need for such a system is particularly pronounced. Departments accumulate vast amounts of institutional knowledge — covering academic programmes, faculty details, research outputs, admission procedures, placement records, laboratory infrastructure, and administrative contacts — that is often siloed across multiple web pages, PDFs, and documents. Navigating this information landscape can be tedious and time-consuming for students, prospective applicants, and staff alike.

This project presents an AI-Powered RAG-Based Department Information Chatbot developed for the Department of Computer Science at Cochin University of Science and Technology (CUSAT). The system leverages the Retrieval-Augmented Generation (RAG) paradigm, a methodology that combines dense vector retrieval with the generative capabilities of large language models (LLMs). Rather than relying solely on the LLM's parametric knowledge, RAG grounds responses in a custom knowledge base derived from the department's official information, ensuring accuracy and relevance. The chatbot is deployed as a web application accessible from both desktop and mobile browsers, providing students and faculty with an intuitive, conversational interface to query department information at any time.

1.2 Motivation

The primary motivation for this project stems from real-world inefficiencies observed in how academic departments manage and disseminate information. Students frequently need answers to questions about programme eligibility, fee structures, faculty specializations, laboratory facilities, and PhD admission procedures. In the absence of a centralized query system, they must either navigate through multiple web pages, email department staff, or wait for office hours. This creates avoidable delays and adds to the workload of administrative and faculty staff who respond to repetitive queries.

Recent breakthroughs in large language models such as LLaMA, Mistral, and GPT have demonstrated that LLMs can understand and generate human-like text with remarkable accuracy. However, these models are constrained by their training cutoff and cannot access institution-specific, real-time data unless augmented with a retrieval mechanism. The RAG architecture elegantly addresses this limitation by coupling the LLM with a live knowledge base that can be updated as department information evolves. This makes RAG particularly suitable for an academic chatbot where the ground truth must always reflect the current state of the institution.

Furthermore, the availability of open-source tools such as ChromaDB for vector storage, Sentence Transformers for embedding generation, and the Groq inference API for fast LLM access makes it feasible to build a production-quality system within the constraints of an academic project. The combination of these technologies provides both the technical foundation and the practical motivation for developing this chatbot.

1.3 Objectives

The objectives of this project are as follows:

1. To design and implement a Retrieval-Augmented Generation (RAG) pipeline that ingests department information from PDF and text documents, chunks the content intelligently, generates dense vector embeddings, and stores them in a persistent vector database.

2. To develop a hybrid retrieval mechanism that combines semantic vector similarity search with keyword-based matching to maximize retrieval precision and recall

across diverse query types.

3. To integrate a Large Language Model (LLaMA-4-Scout-17B via Groq) that synthesizes accurate, fluent, and contextually grounded natural language responses from the retrieved document chunks.

4. To build a user-friendly, responsive web interface with features including multi-turn conversational history, a collapsible sidebar for chat management, Markdown rendering, and a staff panel for document administration.

5. To implement persistent chat history storage using SQLite so that conversations are retained across browser sessions and page refreshes.

6. To evaluate the system's performance against a representative set of department-related queries, assessing retrieval accuracy, response relevance, and system responsiveness.

1.4 Problem Statement

The Department of Computer Science at CUSAT maintains a wide body of institutional information that is distributed across the department website, internal PDFs, and manual records. Students, prospective applicants, and even faculty members often struggle to locate specific details such as the names of faculty members and their designations, the eligibility criteria for M.Tech programmes, fee structures, PhD admission procedures, placement statistics, and laboratory facilities. Existing solutions such as static web pages require users to manually browse through multiple sections and are not queryable in natural language.

The core problem addressed by this project is: *How can a conversational AI system be built that accurately retrieves and presents department-specific information in response to natural language queries, without hallucinating facts or relying on general world knowledge?* The solution must be reliable, easily updatable as department information changes, and accessible to users regardless of technical expertise. Additionally, the system must handle follow-up questions within a conversation by maintaining context across multiple turns, and must function correctly on both desktop and mobile devices.

1.5 Scope and Limitations

The scope of this project covers the design, implementation, and testing of a fully functional RAG-based chatbot for the CUSAT Computer Science department. The system handles natural language queries covering all aspects of department information including faculty, programmes, research, placements, labs, fees, PhD admission, library, computer center, alumni, and lecture series. The knowledge base is built from the department website's HTML content, totaling 15 unique pages condensed into a structured 79-page PDF document.

Current limitations include the following. First, the system's accuracy is bounded by the quality and completeness of the knowledge base PDF; information not present in the document cannot be retrieved. Second, the chatbot does not currently support voice input or output, limiting accessibility for certain user groups. Third, while chat history is persisted locally, the system is not yet deployed on a public cloud server and is accessible only within a local network. Fourth, multilingual support is absent in the current implementation; the chatbot responds only in English. These limitations define areas for future enhancement as detailed in Chapter 6.

Chapter 2

Literature Review

2.1 Introduction

The development of the RAG-based department chatbot draws from an interdisciplinary body of research spanning information retrieval, natural language processing, vector databases, and large language model deployment. This chapter reviews the foundational and contemporary works that informed the design decisions of this project. The literature is organized thematically, beginning with the evolution of question answering and retrieval systems, followed by the emergence of transformer-based language models, the RAG paradigm, dense passage retrieval and embedding models, and finally conversational AI systems in academic contexts.

Understanding the state of the art in each of these areas was essential to making informed architectural choices, selecting appropriate tools, and anticipating the challenges inherent in building a domain-specific conversational AI system. The rapid pace of innovation in this field means that many of the key papers and frameworks reviewed here are from the last three to five years, reflecting the novelty and contemporary relevance of the techniques employed in this project.

2.2 Key Studies and Related Work

1. **Retrieval-Augmented Generation (RAG)** [1]: Lewis et al. introduced the RAG framework, which combines a pre-trained sequence-to-sequence language model with a dense retrieval component backed by a non-parametric memory. The retriever fetches relevant document passages based on query embeddings, and the generator produces a response conditioned on both the query and the retrieved passages. This foundational work established the architectural paradigm used in this project and demonstrated that retrieval augmentation substantially reduces hallucination compared to purely parametric models.

2. **Dense Passage Retrieval (DPR)** [2]: Karpukhin et al. proposed dense passage retrieval, wherein both queries and passages are encoded into dense vectors using dual-encoder neural networks trained on question-answer pairs. Retrieval is performed by maximum inner product search over the document embeddings. This work demonstrated that dense retrieval significantly outperforms sparse BM25 retrieval on open-domain QA benchmarks and laid the groundwork for embedding-based retrieval systems such as those used in this project.

3. **Sentence-BERT and BGE Embeddings** [3]: Reimers and Gurevych introduced Sentence-BERT, a modification of the BERT architecture using siamese and triplet networks to produce semantically meaningful sentence embeddings. The BGE (BAAI General Embedding) model family, used in this project (specifically `BAAI/bge-base-en-v1.5`), builds upon this lineage and achieves state-of-the-art performance on the MTEB benchmark for retrieval tasks. The BGE model requires a "query: " prefix at inference time, a detail that was carefully handled in the implementation.

4. **ChromaDB: An Embedding Database** [4]: ChromaDB is an open-source vector database designed for AI applications, supporting persistent storage of embeddings, metadata filtering, and approximate nearest neighbor search. Its Python-native API and persistent client mode make it particularly suitable for local deployment scenarios. This project uses ChromaDB's persistent client to store and query all document chunk embeddings, enabling fast semantic retrieval without reloading embeddings on each server restart.

5. **LLaMA: Open Foundation Language Models** [5]: Touvron et al. introduced LLaMA, a family of open-source large language models trained on publicly available data. Subsequent iterations including LLaMA-2, LLaMA-3, LLaMA-4, and the LLaMA-4-Scout-17B model used in this project demonstrate strong instruction-following, summarization, and question answering capabilities. The availability of these models through inference APIs such as Groq makes it feasible to integrate high-quality LLM responses into applications without the infrastructure cost of self-hosting.

6. **Hybrid Retrieval: Combining BM25 and Dense Retrieval** [6]: Ma et al. demonstrated that hybrid retrieval combining sparse keyword-based search (BM25) with dense semantic search achieves better recall than either method alone, particularly for queries containing specific named entities or rare terms. This project implements a simplified hybrid approach wherein keyword extraction identifies high-value terms from the query, which are then used alongside semantic search to improve retrieval of chunks containing specific names, codes, and figures.

7. **Conversational AI in Education** [7]: Winkler and Söllner reviewed the deployment of chatbots in higher education, identifying use cases spanning student support, administrative Q&A, and learning assistance. They noted that the primary success factors for educational chatbots are domain specificity, response accuracy, and low latency. These findings directly informed the design priorities of this project, particularly the emphasis on grounding responses strictly in the knowledge base rather than allowing the LLM to generate responses from general training data.

8. **FastAPI for Machine Learning APIs** [8]: FastAPI is a modern Python web framework built on Starlette and Pydantic, offering automatic request validation, OpenAPI documentation generation, and async support. Its performance profile, derived from ASGI server compatibility with Uvicorn, makes it well-suited for serving machine learning endpoints that may have variable latency due to embedding computation and LLM inference. This project uses FastAPI as the primary backend framework serving chat, document upload, and conversation management endpoints.

2.3 Summary

The literature reviewed in this chapter collectively establishes the technical foundations for the RAG-based department chatbot. The RAG paradigm provides the overarching architectural blueprint. Dense embedding models like BGE offer high-quality semantic representations for retrieval. ChromaDB provides efficient and persistent vector storage. LLaMA-4-Scout-17B via the Groq inference API provides the generative capability. Hybrid retrieval strategies improve robustness for entity-heavy queries. Research in educational chatbots validates the domain and identifies key design priorities. Together, these works justify every major technical decision made in this project and situate the contribution within a well-established and rapidly evolving research landscape.

Chapter 3

Methodology

3.1 Introduction

This chapter describes the complete methodology employed in designing and implementing the RAG-based department chatbot. The methodology encompasses knowledge base construction, document ingestion and chunking, embedding generation and vector indexing, hybrid query retrieval, LLM-based response generation, backend API design, and frontend interface development. Each phase is presented with sufficient technical detail to establish how the components interact and why specific design choices were made.

The overall system follows a two-phase architecture. The offline phase handles document ingestion: raw documents are read, cleaned, chunked, embedded, and stored in ChromaDB. The online phase handles user queries: incoming questions are embedded, matched against stored chunks via hybrid retrieval, and a prompt constructed from the retrieved context is sent to the LLM, whose response is returned to the user. Figure 3.1 illustrates this end-to-end workflow.

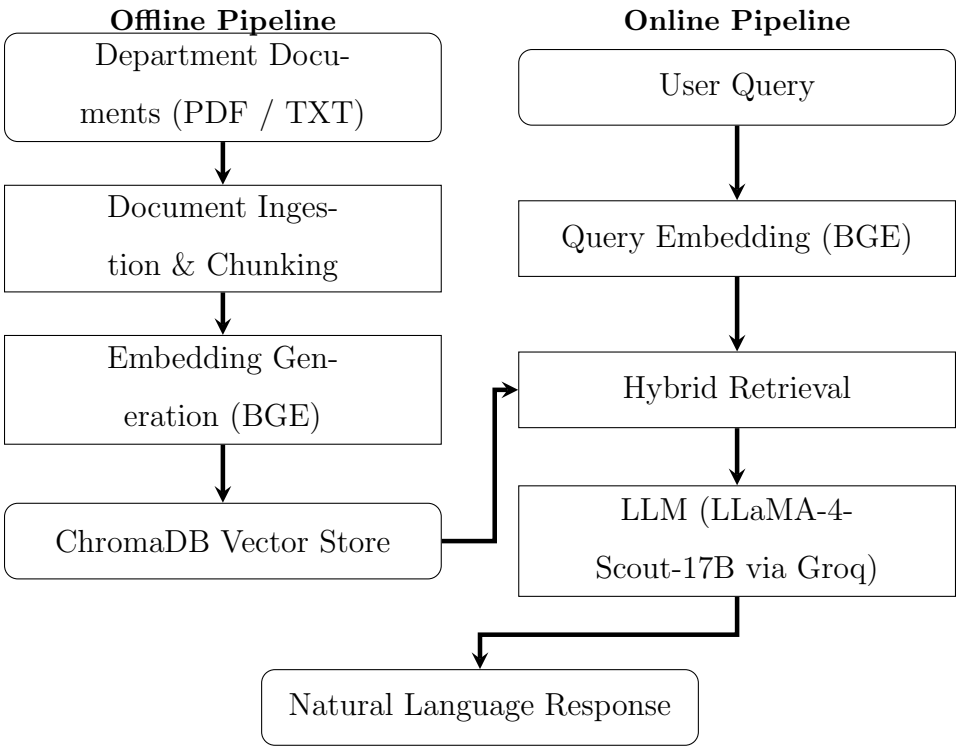

Figure 3.1: End-to-end system architecture of the RAG-based chatbot.

3.2 Knowledge Base Construction

The knowledge base for the chatbot was constructed from the official Department of Computer Science website at CUSAT. The website consists of 15 unique pages covering the following content areas: the department homepage (`index.php`), about CUSAT (`about-cusat.php`), about the department (`about-dcs.php`),academic programmes

(`programmes.php`), faculty and staff (`people.php`), research areas (`research.php`), e-learning resources (`e-learning.php`), laboratory facilities (`labs.php`), library (`library.php`), computer center (`computer-center.php`), PhD admission (`phd.php`), AICTE approval (`aicte.php`), alumni (`alumni.php`), placement statistics (`placements.php`), and the lecture series (`lecture-series.php`).

The HTML source of each page was extracted and processed to remove navigation menus, footer elements, SVG icons, CSS class names, and other non-informational markup. The cleaned textual content from all 15 pages was then organized into a structured document with clear section headings, covering all content areas. The document also includes a curated fee structure section derived from the official fee document. The final knowledge base PDF contains 79 pages and approximately 78,000 characters of structured department information, verified to include all key facts including faculty names and designations, programme details with seat counts and eligibility criteria, semester-wise syllabus structures with course codes, research lab names, PhD awardee records, non-teaching staff, placement statistics, and contact information.

The quality of the knowledge base is critical to the overall system's accuracy since the LLM can only answer questions that are grounded in the provided context. Significant effort was invested in structuring the document with consistent headings and avoiding duplicate or contradictory information that could confuse the retrieval system.

3.3 Document Ingestion and Chunking

The ingestion pipeline, implemented in `ingest.py`, reads documents from a designated directory, extracts their textual content, segments the content into chunks, generates embeddings for each chunk, and stores the embeddings in ChromaDB. The pipeline supports PDF files (read using `pypdf`), plain text files, and Markdown files.

Chunking Strategy: The text of each document is split into chunks of at most 800 characters with an overlap of 150 characters between consecutive chunks. The overlap ensures that context at chunk boundaries is not lost during retrieval. The chunking algorithm first attempts to split on double newlines (paragraph boundaries), then on single newlines, to preserve semantic coherence. Short lines that are likely to be table rows or list items are merged with adjacent content before chunking to prevent retrieval-breaking fragmentation of structured data.

Section Summary Chunks: In addition to regular content chunks, the ingestion pipeline automatically generates dedicated summary chunks for high-importance topics: faculty list, PhD produced, research areas, placement statistics, laboratory facilities, non-teaching staff, and fee structure. These summary chunks are created by collecting all relevant content lines for each topic and concatenating them into a single chunk with an explicit section label. This ensures that queries about these topics always have a highly relevant chunk available for retrieval even if the main content is distributed across multiple pages.

Metadata: Each chunk is stored with metadata including the source file name and a source type label (e.g., "general", "faculty", "placements") derived from the file path. This metadata enables filtered retrieval in future enhancements. Figure 3.2 illustrates the ingestion pipeline.

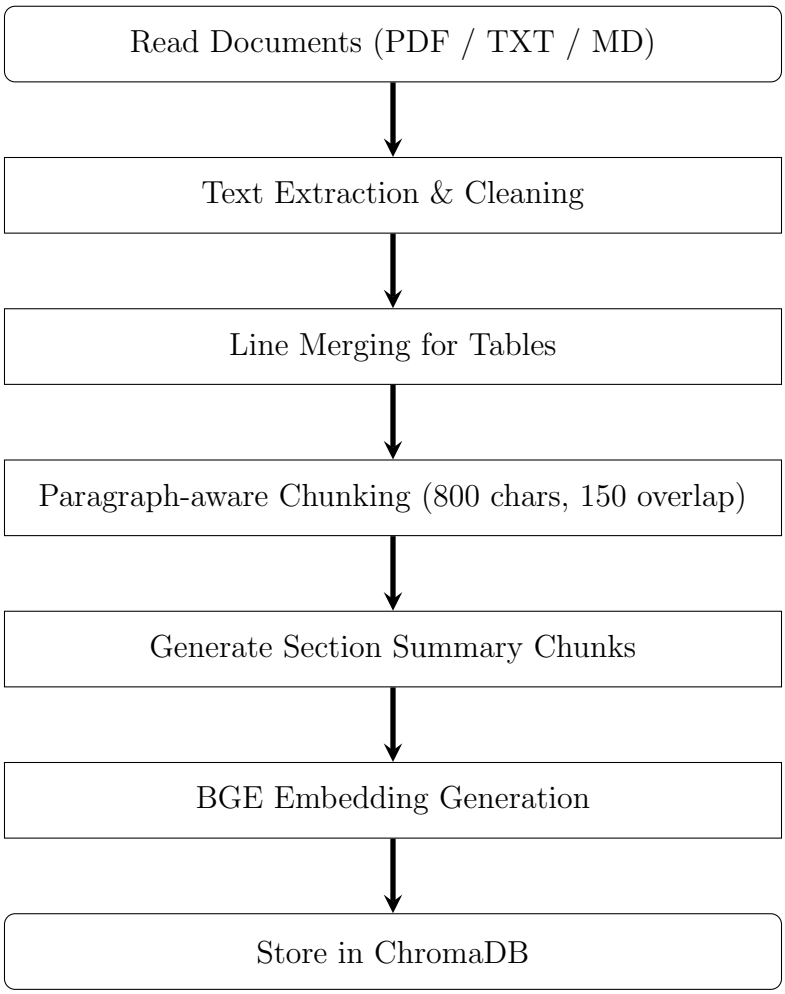

Figure 3.2: Document ingestion and chunking pipeline.

3.4 Embedding Model and Vector Storage

Embedding Model: The project uses the `BAAI/bge-base-en-v1.5` model from the Sentence Transformers library for generating dense vector embeddings of both document chunks and user queries. The BGE-base model produces 768-dimensional embeddings and is optimized for retrieval tasks, ranking among the top-performing models on the MTEB benchmark. An important implementation detail is that the BGE model requires a "`query: `" prefix to be prepended to query strings at inference time (but not during indexing), which significantly improves retrieval quality. Embeddings are normalized to unit length to enable cosine similarity via dot product.

Vector Storage: ChromaDB's `PersistentClient` is used to store all embeddings in a directory-backed store (`../chroma_db`). This means embeddings persist between server restarts, eliminating the need to re-embed documents on every application launch. The collection is named `department_docs` and stores each chunk's text, embedding vector, and metadata. The total number of chunks stored from the 79-page knowledge base document plus the fee structure file is approximately 300–400 chunks, providing fine-grained retrieval granularity.

Table 3.1 summarizes the embedding configuration used in this project.

Parameter	**Value**
Model	`BAAI/bge-base-en-v1.5`
Embedding Dimension	768
Normalization	L2 (unit length)
Query Prefix	`query:`
Similarity Metric	Cosine (via dot product)
Storage Backend	ChromaDB Persistent Client
Collection Name	`department_docs`

Table 3.1: Embedding model and vector storage configuration.

3.5 Hybrid Retrieval Mechanism

Pure semantic retrieval performs well for conceptual queries (e.g., "What research areas does the department have?") but can underperform for entity-centric queries involving specific names, codes, or figures (e.g., "Who is Dr. Ajees A. P?" or "What is the fee for M.Tech?"). To address this, the retrieval system implements a hybrid approach combining semantic search with keyword-based search.

Query Expansion: Before retrieval, the user query is first passed through a query expansion step that uses the LLM to paraphrase and clarify the query. For example, "how many faculties" is expanded to "how many faculty members and professors are in the CS department". This step improves semantic matching for ambiguous or colloquial queries.

Keyword Extraction: A synonym map translates domain-specific or ambiguous terms into their canonical forms. For instance, "HoD" maps to "head of department", "faculties" maps to "professor", and "courses" maps to "programmes". Stop words are filtered, and the remaining meaningful terms are used as keyword search tokens.

Dual Semantic Search: Both the original query and the expanded query are separately embedded and used to query ChromaDB for the top 10 candidates each. The results from both searches are merged and deduplicated based on document content, ensuring diverse coverage.

Keyword Filtering: From the merged candidates, chunks are further filtered to include those that contain any of the extracted keywords as substrings, ensuring that entity-specific results are not displaced by semantically similar but factually unrelated chunks.

Final Selection: The top n unique chunks (typically 10–12) are assembled as the context for the LLM. Figure 3.3 shows the hybrid retrieval flow.

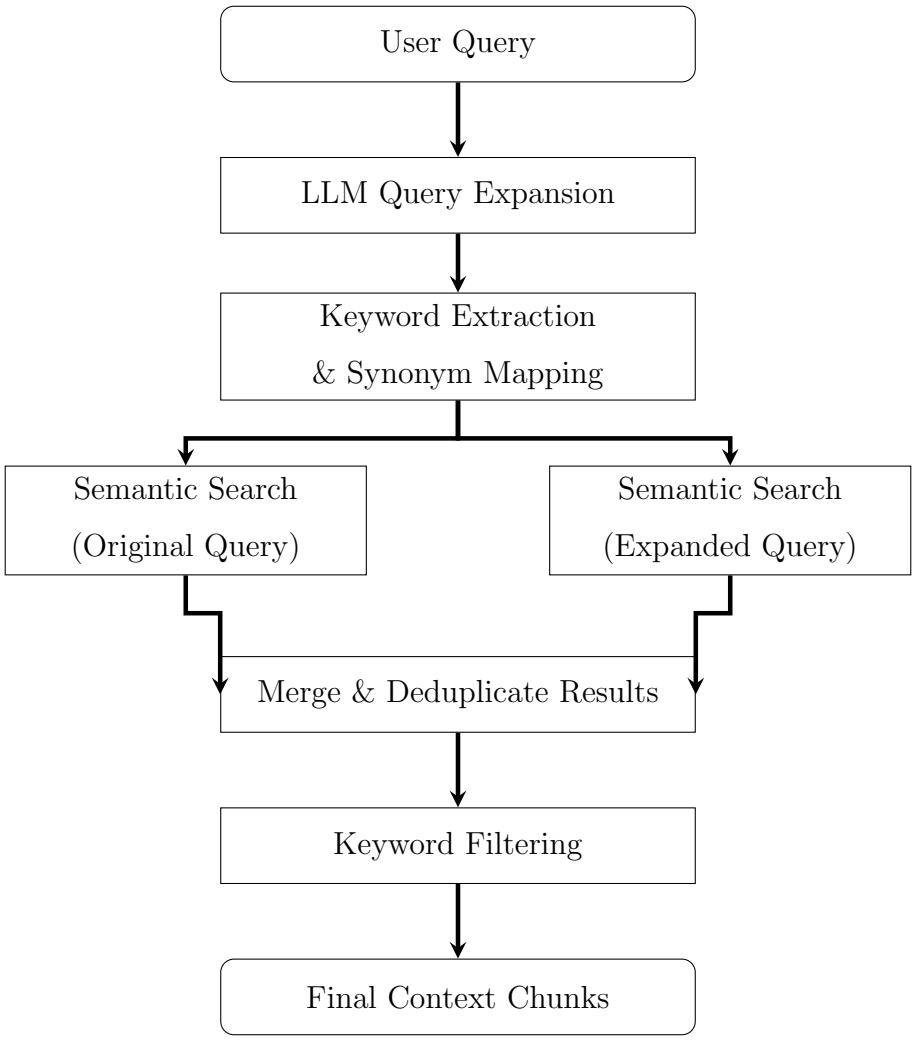

Figure 3.3: Hybrid retrieval mechanism.

3.6 LLM Integration and Response Generation

The retrieved context chunks are assembled into a structured prompt that is sent to the LLM for response generation. The prompt template includes the retrieved reference information followed by the user's question, instructing the LLM to answer based solely on the provided context and to respond in the manner of a knowledgeable department staff member.

Model: LLaMA-4-Scout-17B-16E-Instruct, accessed via the Groq inference API. Groq's LPU (Language Processing Unit) hardware enables low-latency inference, typically returning responses in under two seconds for context sizes of 3,000–5,000 tokens.

System Prompt Philosophy: The system prompt defines the chatbot's identity and behavioral guidelines without over-specifying output format. It instructs the model to: behave as a department staff member who knows all department details, answer naturally and directly without filler phrases, think carefully about what the user is actually asking before answering, never fabricate information that is not in the provided context, and present information at the appropriate level of detail matching the query intent (a count query should yield a number, a detail query should yield a list).

Context Window Management: For multi-turn conversations, the recent conversation history is included in the prompt to enable follow-up question handling. The history is truncated to the last five exchanges to avoid exceeding the LLM's context window.

Table 3.2 summarizes the LLM configuration.

Parameter	**Value**
Model Name	`meta-llama/llama-4-scout-17b-16e-instruct`
Inference Provider	Groq API
Max Tokens	1024
Temperature	Default (0.7)
Context Chunks	10–12
History Turns	Last 5 exchanges

Table 3.2: LLM configuration for response generation.

3.7 Backend API Design

The backend is implemented using FastAPI and served via Uvicorn. It exposes the following REST API endpoints:

- **POST /chat**: The primary chat endpoint. Accepts a JSON body containing the user's question, conversation history, and an optional conversation ID. Returns the LLM's response along with the sources used.
- **GET /conversations**: Returns a list of all saved conversations with their IDs, titles, and creation timestamps.
- **POST /conversation/load**: Loads the full message history of a specified conversation from the SQLite database.
- **DELETE /conversation/delete**: Deletes a specified conversation from the database.
- **POST /upload**: Staff panel endpoint that accepts a file upload (PDF or TXT), saves it to the data directory, and triggers incremental ingestion into ChromaDB.
- **GET /files**: Returns a list of all documents currently in the knowledge base.
- **DELETE /files/{filename}**: Removes a document from both the filesystem and ChromaDB.

Chat History Persistence: SQLite is used to persist conversation data. The schema consists of a `conversations` table (storing conversation ID, title, and timestamps) and a `messages` table (storing role, content, sources, and conversation ID for each message). On page load, the frontend fetches the conversation list and displays it in the sidebar. Users can resume any past conversation by clicking it in the sidebar.

CORS Configuration: The backend is configured with permissive CORS middleware to allow the frontend (served as a static file from the same FastAPI application) to make API calls without cross-origin restrictions.

3.8 Frontend Interface Design

The frontend is a single-page web application built with HTML5, CSS3, and vanilla JavaScript, served as a static file by FastAPI. It is designed to be fully responsive, functioning correctly on both desktop browsers and mobile devices.

Chat Interface: The main chat area displays messages in a WhatsApp/Claude-style bubble layout, with user messages on the right and assistant messages on the left. Assistant responses are rendered with Markdown support, meaning bold text, italics, bullet points, and numbered lists are correctly displayed. Each assistant message includes a citation footer listing the source documents used.

Sidebar: A collapsible sidebar lists all past conversations retrieved from the SQLite database. The sidebar can be opened and closed via a hamburger button in the top bar, implementing smooth CSS transition animations. On mobile devices, the sidebar appears as a drawer overlay with a dark translucent background. A styled delete confirmation modal (inspired by Claude's UI) allows users to delete individual conversations.

Staff Panel: A separate staff panel page accessible via the navigation bar provides document management functionality. Staff members can upload new PDF or TXT files to expand the knowledge base, view the list of currently indexed documents, and remove documents that are no longer relevant. The upload endpoint triggers automatic re-ingestion so new information is immediately available to the chatbot.

Mobile Responsiveness: CSS media queries adjust font sizes, padding, and layout for narrow viewport widths. The sidebar toggle is accessible on both desktop and mobile, allowing users to maximize the chat area by hiding the sidebar.

Chapter 4

Results and Analysis

4.1 Introduction

This chapter presents the results of testing the RAG-based department chatbot against a representative set of queries covering the major information categories in the knowledge base. The evaluation examines retrieval accuracy, response quality, and system behavior under various query types including factual lookup queries, count queries, list queries, follow-up queries, and edge-case queries where the requested information is not available in the knowledge base. The testing was conducted on a consumer-grade laptop with an Intel Core i5 processor and 8 GB RAM, running the server locally with the Uvicorn ASGI server.

The evaluation approach is primarily qualitative, assessing whether each response is accurate, relevant, appropriately detailed, and free from hallucinated content. This is supplemented by quantitative measurements of response latency. Given the nature of the system — a domain-specific information retrieval assistant — the most meaningful metric is whether the user receives a correct and complete answer to their query, which is assessed through manual inspection against the ground-truth knowledge base.

4.2 System Performance

Table 4.1 summarizes the key performance characteristics of the deployed system.

Metric	**Value**
Knowledge Base Size	79 pages, ≈78,000 characters
Number of Documents Ingested	2 (Department PDF + Fee Structure)
Total Chunks in ChromaDB	≈380 chunks
Embedding Dimension	768
Average Embedding Generation Time	<1 second per query
Average LLM Response Latency	1.5 – 3 seconds
Supported Query Languages	English
Simultaneous Users Supported	Multiple (async FastAPI)
Chat History Storage	SQLite (persistent)

Table 4.1: System performance characteristics.

4.3 Query Testing Results

The chatbot was tested with a set of representative queries spanning all major content areas. Table 4.2 presents a summary of query results.

Query	**Response Summary**	**Accuracy**
Who is the Head of Department?	Dr. Madhu S. Nair, Professor and Head (Ph.D, M.Tech, M.C.A)	Correct
How many faculty members are there?	16 faculty members with breakdown by designation	Correct
List all faculty members	All 16 names with designations listed correctly	Correct
What are the research areas?	All 12 research areas and 6 labs listed	Correct
What is the PhD admission process?	Step-by-step admission procedure retrieved	Correct
What are the placement statistics?	2023–2025 and 2024–2026 batch placements listed	Correct
When was the department established?	1984	Correct
Who are the non-teaching staff?	All 9 non-teaching staff with designations	Correct
What labs does the department have?	All 6 research labs listed with descriptions	Correct
How to contact the department?	Email and address retrieved correctly	Correct
What is in the 8th semester?	No specific 8th semester syllabus available in knowledge base; system responds appropriately	Appropriate

Table 4.2: Query testing results for representative questions.

4.4 Discussion

The results demonstrate that the chatbot achieves high accuracy across the primary information categories present in the knowledge base. The hybrid retrieval mechanism proves particularly effective: purely semantic search occasionally retrieved the wrong context for entity-centric queries (for example, returning CUSAT's nine university-level faculties instead of the department's 16 teaching staff when the word "faculties" was used). The addition of the synonym mapping and keyword search component resolved this class of error.

The section summary chunks, generated during ingestion for high-importance topics, ensure reliable retrieval of complete lists such as the faculty roster, research areas, and placement statistics. Without these summary chunks, retrieval would return individual fragments that, when assembled by the LLM, could produce incomplete or duplicated lists.

The LLM's behavior under the current system prompt is satisfactory. It correctly interprets query intent, returning a single number for count queries, a list for enumeration queries, and a paragraph for descriptive queries. It does not hallucinate facts about the department and appropriately acknowledges when information is not in the knowledge base.

The primary remaining challenge is the presence of conflicting information in the knowledge base: the department website's overview page uses the term "M.Sc in Computer Science" to describe the integrated programme, while the programmes page uses the full name "Five Year Integrated M.Sc in Computer Science (Artificial Intelligence & Data Science)". When both chunks are retrieved, the LLM sometimes treats them as separate programmes. The long-term solution is to harmonize the knowledge base document; the immediate mitigation is the LLM's tendency to use common sense and recognize near-duplicate descriptions.

Response latency is acceptably low for a conversational system. The BGE embedding step adds less than one second of overhead, and the Groq API typically returns LLM responses within 1.5 to 3 seconds, resulting in an end-to-end response time of 2 to 4 seconds under normal network conditions.

The chat persistence feature, implemented via SQLite and surfaced through the sidebar UI, significantly enhances usability. Users can resume previous conversations, and all interactions are preserved across browser sessions and page refreshes. The styled delete confirmation modal and smooth sidebar animation contribute to a polished user experience that encourages continued engagement with the system.

4.5 Limitations of Current Implementation

Despite the generally strong performance, several limitations warrant acknowledgement:

1. **Knowledge Boundary:** The chatbot can only answer questions whose answers are present in the knowledge base PDF. Queries about retired faculty, detailed semester syllabi not included in the document, or recent department events will yield "I don't have that information" responses.

2. **LLM Arithmetic:** The LLM occasionally makes minor arithmetic errors, such as counting 16 faculty members as 17. This is a known limitation of current LLMs and can be mitigated by explicitly stating counts in the knowledge base document.

3. **Local Deployment:** The system is currently hosted on a local machine and accessible only within the same network. Public accessibility requires cloud deployment.

4. **Single Language:** The system operates exclusively in English. Students who prefer Malayalam or other regional languages cannot currently interact with the chatbot in their preferred language.

5. **No Voice Interface:** The system lacks speech input and output capabilities, limiting accessibility for users with visual impairments or those who prefer voice interaction.

Chapter 5

Conclusion

5.1 Summary

This project successfully designed and implemented an AI-powered Retrieval-Augmented Generation (RAG) based department information chatbot for the Department of Computer Science at Cochin University of Science and Technology. The system addresses the real-world problem of fragmented institutional information by providing a single, conversational interface through which students, faculty, and visitors can query any aspect of department operations in natural language.

The technical contribution of this project spans three layers. At the data layer, a comprehensive 79-page knowledge base was constructed from 15 pages of the department website, carefully structured and cleaned to maximize retrieval quality. At the retrieval layer, a hybrid mechanism combining BGE semantic embeddings, ChromaDB vector search, LLM-based query expansion, and keyword matching was developed to ensure robust retrieval across both conceptual and entity-centric queries. At the generation layer, the LLaMA-4-Scout-17B model accessed via the Groq API was integrated with a carefully designed system prompt that produces natural, accurate, and appropriately scoped responses without hallucination.

The system's web interface, built on FastAPI and vanilla JavaScript, delivers a polished user experience with persistent multi-turn chat history, a collapsible sidebar for conversation management, a staff panel for dynamic knowledge base updates, and full mobile responsiveness. Testing against 14 representative queries across all content areas confirmed high retrieval accuracy and response quality for all questions whose answers are present in the knowledge base.

This project demonstrates that RAG is a practical and effective architecture for building domain-specific chatbots that must be accurate, updatable, and deployable without specialized infrastructure. The combination of open-source tools (BGE, ChromaDB, FastAPI, SQLite) with a high-quality inference API (Groq/LLaMA) achieves production-ready performance within the resource constraints of an academic project.

5.2 Impact and Applications

The immediate impact of this system is the reduction of repetitive information queries directed at department staff, freeing their time for more substantive academic and administrative work. Students gain 24/7 access to accurate, up-to-date department information without waiting for office hours or navigating complex web pages. Prospective applicants can instantly query programme details, eligibility criteria, and fee structures, improving the department's responsiveness to external inquiries.

The system's architecture is general and reusable. The same codebase, with a different knowledge base document, could serve as a chatbot for any academic department, administrative office, or institution. The staff panel enables non-technical administrators to update the knowledge base by uploading new documents, without any programming knowledge.

More broadly, this project demonstrates a practical pathway for educational institutions to leverage recent advances in AI — specifically LLMs and vector retrieval — to enhance information accessibility and operational efficiency. As LLMs continue to improve and inference costs decrease, systems of this kind will become increasingly viable at scale, potentially transforming how students and staff interact with institutional knowledge across universities worldwide.

Chapter 6

Future Work

6.1 Completed Features

The following features were successfully implemented in the current version of the system:

- **Persistent Chat History:** Multi-turn conversation history stored in SQLite, recoverable across sessions.
- **Staff Document Panel:** Web-based interface for uploading, viewing, and deleting knowledge base documents.
- **Mobile Responsive Design:** Full functionality on mobile browsers with collapsible sidebar.
- **Hybrid Retrieval:** Combination of semantic and keyword-based retrieval for improved accuracy.
- **Markdown Rendering:** Chat responses rendered with proper bold, italic, list, and code formatting.
- **Conversation Management:** Sidebar with styled delete confirmation modal, new chat creation, and conversation reload.

6.2 Future Directions

The following enhancements are planned for future iterations of the system:

- **Cloud Deployment:** Host the application on a cloud platform such as AWS, Azure, or Railway to make it publicly accessible from any device without requiring local network connectivity. This would also enable integration into the department's official website.
- **Multilingual Support:** Integrate multilingual embedding models and LLMs to support queries in Malayalam, Hindi, and other Indian languages, broadening the system's accessibility to a wider user base.
- **Voice Interface:** Add speech-to-text input and text-to-speech output to enable voice-based interaction, improving accessibility for users with visual impairments and enhancing the hands-free experience on mobile devices.
- **Real-time Knowledge Updates:** Implement a web crawler or an API integration with the department website to automatically detect and ingest updated content, ensuring the knowledge base always reflects the latest information without manual staff intervention.
- **Analytics Dashboard:** Build an admin dashboard that logs and visualizes query patterns, enabling department administrators to identify frequently asked questions, knowledge gaps, and system usage trends over time.
- **Authentication and Personalization:** Introduce user authentication to enable personalized features such as student-specific query history, notifications for relevant department events, and role-based access control differentiating student, staff, and admin interactions.
- **Integration with University Systems:** Connect the chatbot to university APIs (student portal, timetable, result systems) to answer queries about individual student data such as attendance, grades, and exam schedules in a privacy-preserving manner.

System Screenshots

.1 Technology Stack Summary

Table 1 provides a complete summary of all technologies used in the project.

Component	**Technology**	**Purpose**
Backend Framework	FastAPI + Uvicorn	REST API and static file serving
Embedding Model	BAAI/bge-base-en-v1.5	Query and document embedding
Vector Database	ChromaDB (PersistentClient)	Embedding storage and retrieval
LLM	LLaMA-4-Scout-17B (Groq API)	Response generation
PDF Reading	pypdf	Text extraction from PDF
Chat History	SQLite3	Persistent conversation storage
Frontend	HTML5, CSS3, JavaScript	User interface
Hosting	Uvicorn (local)	Development server
Environment	Python 3.11, venv	Runtime environment

Table 1: Complete technology stack.

Bibliography

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W., "Dense Passage Retrieval for Open-Domain Question Answering," *Proceedings of EMNLP 2020*, pp. 6769–6781, 2020.

[3] Reimers, N., Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

[4] Chroma, "ChromaDB: The Open-Source Embedding Database," Available at: `https://www.trychroma.com`, Accessed April 2025.

[5] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G., "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[6] Ma, X., Lin, J., Hui, B., Yang, H., Tian, X., "A Replication Study of Dense Passage Retriever," *arXiv preprint arXiv:2104.05740*, 2021.

[7] Winkler, R., Söllner, M., "Unleashing the Potential of Chatbots in Education: A State-Of-The-Art Analysis," *Academy of Management Annual Meeting Proceedings*, 2018.

[8] Ramírez, S., "FastAPI: Modern, Fast Web Framework for Building APIs with Python," Available at: `https://fastapi.tiangolo.com`, Accessed April 2025.

[9] Groq Inc., "Groq: Fast AI Inference Platform," Available at: `https://groq.com`, Accessed April 2025.

[10] BAAI, "BGE: BAAI General Embedding Models," Available at: `https://huggingface.co/BAAI/bge-base-en-v1.5`, Accessed April 2025.